Authentication Login Call Trace

Google and GitHub OAuth in the Product Engineering Workbench - from a signed-out home page to an owner-scoped Project view.

Purpose

This document traces the actual first-slice login path. It names the repository file or function responsible at each application-owned step and labels the Better Auth and provider-owned work that sits outside the repository.

Reading the trace

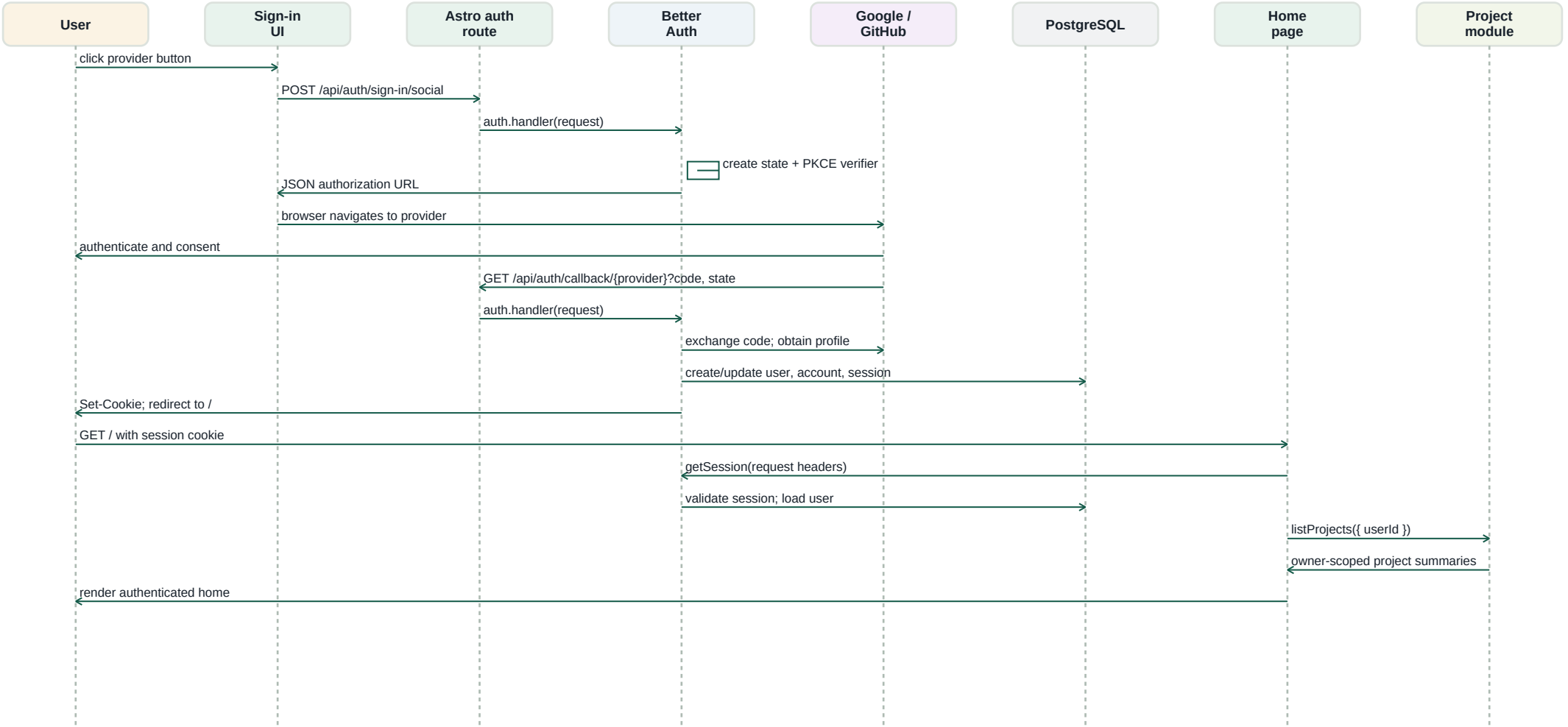
Layer	Responsibility
Repository	Astro pages/routes, React islands, composition, Better Auth adapter, principal resolution, and Project module calls.
Better Auth	OAuth state and PKCE generation, provider authorization URL, callback validation, identity/session persistence, signed cookie, and redirect.
Google or GitHub	User authentication, consent, code issuance, and the provider-side token/profile response.
PostgreSQL	Durable Better Auth user, account, session, and verification records in the auth schema.

Key security boundary

The browser chooses a provider but never supplies trusted ownership. Only after Better Auth validates the provider callback and session does the application produce a Principal. The Project module receives that server-derived userId for owner-scoped reads and commands.

Sequence diagram

The green and blue application path is shared by Google and GitHub. The provider switch is the single variable supplied by the sign-in button.



The initial POST returns an authorization URL as JSON. The React island deliberately performs the browser navigation, so the external provider is reached only after the application has received that URL.

Explained call trace - initiating OAuth

1. Render the signed-out home page

GET / constructs the server composition and asks Better Auth for the current session. With no valid cookie, resolvePrincipal returns null and the page renders the SignInIsland.

`Repository: src/pages/index.astro lines 7-14; src/adapters/better-auth/principal-resolver.ts lines 12-18`

2. Choose a social provider

The user clicks Continue with Google or Continue with GitHub. The signIn function accepts only those two provider identifiers, sends JSON to the same-origin Better Auth route, and requests a post-login callbackURL of /.

`Repository: src/islands/sign-in-island.tsx lines 5-10 and 15`

3. Forward the request to Better Auth

Astro's catch-all route owns no OAuth policy. It forwards the original Request to the configured Better Auth handler. serverComposition validates runtime configuration, creates database connections, and builds the configured auth instance.

`Repository: src/pages/api/auth/[...all].ts line 4; src/composition/server-composition.ts lines 12-19; src/composition/runtime-config.ts lines 19-21`

4. Create the provider authorization URL

The Better Auth adapter supplies the base URL, signing secret, trusted origin, PostgreSQL/Kysely connection, and provider credentials. Better Auth then creates OAuth state and a PKCE verifier, builds the provider authorization URL, and returns it as JSON.

`Repository: src/adapters/better-auth/server-auth.ts lines 6-18`

`Library: node_modules/better-auth/dist/api/routes/sign-in.mjs lines 107-209`

Explained call trace - provider and callback

5. Authenticate with Google or GitHub

The island reads the returned url and uses window.location.assign. The browser leaves the application. The provider authenticates the user and may request consent; this provider-side work is outside the repository.

`Repository: src/islands/sign-in-island.tsx lines 8-10`

6. Receive and validate the OAuth callback

Google or GitHub redirects the browser to /api/auth/callback/google or /api/auth/callback/github with an authorization code and state. The same Astro catch-all forwards that request to Better Auth. Better Auth parses state, retrieves the PKCE verifier, exchanges the code, and obtains the provider profile.

`Repository: src/pages/api/auth/[...all].ts line 4`

`Library: node_modules/better-auth/dist/api/routes/callback.mjs lines 21-96`

7. Persist identity and session

Better Auth requires a provider email, creates or updates local identity/account state as appropriate, creates a session, and records the result in PostgreSQL. The application migration defines the auth user, session, account, and verification tables.

`Repository: migrations/20260809000100-first-slice.sql lines 11-58`

`Library: node_modules/better-auth/dist/api/routes/callback.mjs lines 133-169`

8. Set the signed cookie and redirect home

Better Auth signs the session token using BETTER_AUTH_SECRET, writes the session cookie, and redirects to the callbackURL supplied in step 2: /. The browser now includes that cookie on the next request.

`Repository: src/adapters/better-auth/server-auth.ts lines 9-12`

`Library: node_modules/better-auth/dist/cookies/index.mjs lines 122-135; callback.mjs lines 165-176`

Explained call trace - authenticated application

9. Resolve the authenticated Principal

On the redirected GET /, resolvePrincipal calls auth.api.getSession using the request headers. It converts the validated Better Auth user into the app's AuthenticatedPrincipal: userId for authorization, plus name and email for presentation.

`Repository: src/adapters/better-auth/principal-resolver.ts lines 3-18; src/pages/index.astro lines 7-9`

10. Load the owner-scoped Project home

The home page passes the server-derived Principal to projects.listProjects. The Project module therefore receives trusted identity rather than a browser-submitted owner ID, and the page renders the authenticated home and account controls.

`Repository: src/pages/index.astro lines 8-14; src/composition/server-composition.ts lines 17-19`

Resulting trust chain

Provider assertion -> Better Auth callback validation -> durable session -> signed cookie -> server-side session lookup -> application Principal -> owner-scoped Project access. The browser never determines Project ownership.